

## Plan de Tests : Classe Commande

### Tests du constructeur et getters

| Méthode de test        | Actions                                         | Attendu                                                                                                                                                        |
|------------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| testConstructeurValide | Crée une commande avec un client valide         | La commande est créée, le client est correctement associé, les listes de pizzas et de quantités sont vides, l'état est CREEE et la date/heure est initialisée. |
| testGetClient          | Appelle getClient() sur une commande créée      | Le client retourné est celui passé au constructeur.                                                                                                            |
| testGetPizzasVide      | Appelle getPizzas() juste après la création.    | La liste des pizzas est vide.                                                                                                                                  |
| testGetQuantitesVide   | Appelle getQuantites() juste après la création. | La liste des quantités est vide.                                                                                                                               |
| testGetEtatInitial     | Appelle getEtat() après création.               | L'état de la commande est CREEE.                                                                                                                               |
| testGetDateHeure       | Appelle getDateHeure() après création.          | La date et l'heure ne sont pas nulles                                                                                                                          |

### Tests des setters

| Méthode de test    | Actions                                   | Attendu                                   |
|--------------------|-------------------------------------------|-------------------------------------------|
| testSetEtatValide  | Modifie l'état de la commande en VALIDEE. | L'état de la commande devient VALIDEE.    |
| testSetEtatTraitee | Modifie l'état de la commande en TRAITEE. | Modifie l'état de la commande en TRAITEE. |

### Tests des méthodes logiques

#### ➤ Ajout de pizzas

| Méthode de test        | Actions                                         | Attendu                                                                        |
|------------------------|-------------------------------------------------|--------------------------------------------------------------------------------|
| testAjouterPizzaValide | Ajoute une pizza avec une quantité positive.    | La pizza est ajoutée à la liste et la quantité correspondante est enregistrée. |
| testAjouterPizzaNull   | Ajoute une pizza avec une quantité égale à 0.   | La pizza n'est pas ajoutée à la commande.                                      |
| testAjouterPizzaNull   | Ajoute une pizza null avec une quantité valide. | Aucun ajout n'est effectué.                                                    |

➤ **Calcul du prix total**

| Méthode de test                 | Actions                                             | Attendu                                                  |
|---------------------------------|-----------------------------------------------------|----------------------------------------------------------|
| testGetPrixTotalUnePizza        | Ajoute une pizza avec quantité 2 et prix 10 €.      | Le prix total retourné est 20.0.                         |
| testGetPrixTotalPlusieursPizzas | Ajoute plusieurs pizzas avec différentes quantités. | Le prix total correspond à la somme correcte des pizzas. |
| testGetPrixTotalCommandeVide    | Calcule le prix total sans pizza.                   | Le prix total est 0.0.                                   |

➤ **Méthodes equals() et hashCode()**

| Méthode de test                | Actions                                                           | Attendu                              |
|--------------------------------|-------------------------------------------------------------------|--------------------------------------|
| testEqualsTrue                 | Compare deux commandes avec le même client et la même date/heure. | Les deux commandes sont égales.      |
| testEqualsFalseClientDifférent | Compare deux commandes avec des clients différents.               | Les deux commandes sont différentes. |
| testHashCode                   | Compare le hashCode de deux commandes identiques.                 | Les hashCodes sont identiques.       |

➤ **Méthode toString()**

| Méthode de test | Actions                                          | Attendu                                                                                                     |
|-----------------|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| testToString    | Appelle toString() sur une commande avec pizzas. | La chaîne retournée contient l'email du client, les pizzas, l'état de la commande et le prix total formaté. |

## Plan de Tests : Classe Pizza

### Tests du constructeur et getters

| Méthode de test        | Actions                                         | Attendu                                                                                                                                                        |
|------------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| testConstructeurValide | Crée une commande avec un client valide.        | La commande est créée, le client est correctement associé, les listes de pizzas et de quantités sont vides, l'état est CREEE et la date/heure est initialisée. |
| testGetClient          | Appelle getClient() sur une commande créée.     | Le client retourné est celui passé au constructeur.                                                                                                            |
| testGetPizzasVide      | Appelle getPizzas() juste après la création.    | La liste des pizzas est vide.                                                                                                                                  |
| testGetQuantitesVide   | Appelle getQuantites() juste après la création. | La liste des quantités est vide.                                                                                                                               |
| testGetEtatInitial     | Appelle getEtat() après création.               | L'état de la commande est CREEE.                                                                                                                               |
| testGetDateHeure       | Appelle getDateHeure() après création.          | La date et l'heure ne sont pas nulles.                                                                                                                         |

### 2. Tests des setters

| Méthode de test    | Actions                                   | Attendu                                |
|--------------------|-------------------------------------------|----------------------------------------|
| testSetEtatValide  | Modifie l'état de la commande en VALIDEE. | L'état de la commande devient VALIDEE. |
| testSetEtatTraitee | Modifie l'état de la commande en TRAITEE. | L'état de la commande devient TRAITEE. |

### 3. Tests des méthodes logiques

#### > Ajout de pizzas

| Méthode de test              | Actions                                         | Attendu                                                                        |
|------------------------------|-------------------------------------------------|--------------------------------------------------------------------------------|
| testAjouterPizzaValide       | Ajoute une pizza avec une quantité positive.    | La pizza est ajoutée à la liste et la quantité correspondante est enregistrée. |
| testAjouterPizzaQuantiteZero | Ajoute une pizza avec une quantité égale à 0.   | La pizza n'est pas ajoutée à la commande.                                      |
| testAjouterPizzaNull         | Ajoute une pizza null avec une quantité valide. | Aucun ajout n'est effectué.                                                    |

➤ **Calcul du prix total**

| Méthode de test                 | Actions                                             | Attendu                                                  |
|---------------------------------|-----------------------------------------------------|----------------------------------------------------------|
| testGetPrixTotalUnePizza        | Ajoute une pizza avec quantité 2 et prix 10 €.      | Le prix total retourné est 20.0.                         |
| testGetPrixTotalPlusieursPizzas | Ajoute plusieurs pizzas avec différentes quantités. | Le prix total correspond à la somme correcte des pizzas. |
| testGetPrixTotalCommandeVide    | Calcule le prix total sans pizza.                   | Le prix total est 0.0.                                   |

➤ **Méthodes equals() et hashCode()**

| Méthode de test                | Actions                                                           | Attendu                              |
|--------------------------------|-------------------------------------------------------------------|--------------------------------------|
| testEqualsTrue                 | Compare deux commandes avec le même client et la même date/heure. | Les deux commandes sont égales.      |
| testEqualsFalseClientDifférent | Compare deux commandes avec des clients différents.               | Les deux commandes sont différentes. |
| testHashCode                   | Compare le hashCode de deux commandes identiques.                 | Les hashCodes sont identiques.       |

➤ **Méthode toString()**

| Méthode de test | Actions                                          | Attendu                                                                                                     |
|-----------------|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| testToString    | Appelle toString() sur une commande avec pizzas. | La chaîne retournée contient l'email du client, les pizzas, l'état de la commande et le prix total formaté. |

## Annexe - Plan de Tests : Classe TestIngredient :

Informations de base :

- Ingrédient basique : instance de Ingredient créée avec le nom « Tomate » et un prix unitaire de 0,5 €.
- Ingrédient complet : instance de Ingredient créée avec le nom « Olive » et un prix unitaire de 0,8 €.

Tests du constructeur et getters :

| Méthode de test             | Actions                                                   | Attendu                                                                      |
|-----------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------|
| testConstructeurValide      | Crée un ingrédient "Tomate" avec un prix de 0.5.          | L'objet est bien créé. Le nom est "Tomate" et le prix est 0.5.               |
| testConstructeurNomVide     | Crée un ingrédient avec un nom vide "" et un prix de 0.8. | Une exception IllegalArgumentException est levée (message contenant "nom").  |
| testConstructeurNomNull     | Crée un ingrédient avec un nom null et un prix de 1.0.    | Une exception IllegalArgumentException est levée (message contenant "nom").  |
| testConstructeurPrixNegatif | Crée un ingrédient "Mozzarella" avec un prix négatif -1.  | Une exception IllegalArgumentException est levée (message contenant "prix"). |

Tests des setters :

| Méthode de test     | Actions                            | Attendu                                                |
|---------------------|------------------------------------|--------------------------------------------------------|
| TestSetPrixValide   | Modifie le prix de "Tomate" à 1.2. | La méthode retourne true et le prix devient 1.2.       |
| TestSetPrixInvalide | Tente de modifier le prix à 0.     | La méthode retourne false et le prix reste 0.5.        |
| TestSetPrixEleve    | Modifie le prix à 1 000 000.       | La méthode retourne true et le prix devient 1 000 000. |

Tests des méthodes logiques :

| Méthode de test        | Actions                                                     | Attendu                                                  |
|------------------------|-------------------------------------------------------------|----------------------------------------------------------|
| testEqualsMemeNom      | Compare deux ingrédients "Tomate" avec des prix différents. | Les deux objets sont égaux (equals retourne true).       |
| testEqualsNomDifferent | Compare "Tomate" et "Olive".                                | Les deux objets sont différents (equals retourne false). |
| testHashCode           | Compare le hashCode de deux ingrédients "Tomate".           | Les hashCodes sont identiques.                           |
| testToStringFormat     | Appelle toString() sur "Tomate".                            | Le texte retourné contient "Tomate" et "€".              |

